SH-CADI — Code Overview (what each block does)

Code: github.com/dkansagara12-alt/SH-CADI

A file-by-file map of the code. For each file you get its job in one line, then each class or function and what it does. Use it to find code quickly when the examiner says "show me where X happens".

Folder map

```
sh_cadi/
├── main.py                    entry point: python -m sh_cadi.main
├── demo.py                    the full 7-section project demo + PASS/FAIL scorecard
├── dedup_system.py            upload / dedup / reconstruct pipeline (ties everything together)
├── chunking/chunker.py        split a file into fixed-size chunks
├── hashing/hasher.py          SHA-256 of a chunk
├── hash_index/hashtable.py    chained hash table (the index)
├── storage/
│   ├── cloud_storage.py       simple simulated cloud: one file per chunk
│   └── protected_store.py     multi-node store with Reed-Solomon fragments
├── merkle/
│   ├── record_digest.py       hash of one index record (what the tree protects)
│   ├── merkle_tree.py         Phase-1 Merkle tree (now the baseline)
│   ├── integrity.py           Phase-1 verify / diagnose (baseline detection)
│   ├── bucket_merkle.py       SH-CADI bucketed fixed-shape Merkle tree
│   └── auditor.py             Detect + Localize
├── recovery/
│   ├── journal.py             sharded, checksummed write-ahead journal
│   ├── healer.py              the self-healing engine (Detect→Localize→Recover→Repair→Verify)
│   ├── baseline.py            conventional full-rebuild recovery
│   ├── erasure.py             Reed-Solomon RS(k,m) over GF(256) + replication
│   └── chunk_healer.py        scrub and repair chunk fragments
├── fault_injection/injector.py controlled, seeded fault injection with a ground-truth log
├── benchmark/                 evaluation harness (systems, worker, runner, report, final demo)
└── visualization/plots.py     the six result plots
tests/                         66 unit tests (test_phase1 … test_phase3b)
benchmark_results/             results.json, RESULTS.md, plots/ (measured on our PC)
docs/                          ALGORITHMS.md, LITERATURE.md, PROJECT_GUIDE.md, this file
```

How the pieces call each other

Upload a file

```
ContentAddressableStorage.upload_file
  → chunk_file → compute_hash
  → SelfHealingIndex.before_write  (verify/heal this record's bucket first)
  → HashTable.search
       found:     ref_count += 1 → HashTable.update
       not found: storage.store_chunk → HashTable.insert
  → SelfHealingIndex.on_upsert → MetadataJournal.append_put → BucketMerkleTree.upsert
```

Heal the index

```
SelfHealingIndex.heal
  → IndexAuditor.detect     (live tree vs stored tree vs trusted root)
  → IndexAuditor.localize   (top-down descent → damaged buckets → changed/missing/extra)
  → MetadataJournal.read_shard + _resolve   (candidates accepted only if digest matches)
  → HashTable.insert/update/delete + BucketMerkleTree.repair_leaves
  → verify buckets + root == trusted_root
```

Heal chunk data

```
SelfHealingIndex.heal_all → heal() (metadata first) → ChunkHealer.heal
  → scrub (classify fragments, check content hash) → decode_verified → rewrite bad fragments → verify
```

Entry points

`main.py`

Block	What it does
`sys.path.insert(...)`	Lets the module run from the project root without installing the package.
`from sh_cadi.demo import main` / `__main__`	Runs the full demo; the exit code is 0 only if every scorecard check passed.

`demo.py` — the final-review demo

Block	What it does
Module docstring	Lists the 7 sections and the run commands (`--quick`, or a record count such as `1000000`).
`ROOT`, `SAMPLE_DIR`, `CHUNK`, `DEPTH`, `CHECKS`	Project root, input folder, 64-byte demo chunks, a tree of depth 6 (64 buckets), and the scorecard list.
`banner`, `check`, `short`, `snapshot`	Print helpers. `check` records a PASS/FAIL. `snapshot` takes the digest of every record, so we can prove the index is identical to its pre-fault state.
`sample_files`	Creates `sample_files/FileA.txt` and `FileB.txt` if they are missing (B shares its first part with A, so dedup has something to find).
`same_file`	Rebuilds a file from its chunks and compares it byte-for-byte with the original.
`build_system`	Fresh store + `SelfHealingIndex` attached to the CAS; uploads A, B, then A again.
`print_heal`	Prints the 5-step heal report (timings, buckets, records examined, sources, locality).
`section_dedup` (1)	Upload stats, sample index entries; checks that a duplicate upload stores 0 new chunks and that reconstruction is exact.
`section_index` (2)	Shows bucket count, the trusted root and the journal shards; checks that a clean system audits as intact.
`section_heal` (3)	Master-prompt example `H → Chunk_X`: inject, heal, show before/after; checks exact localization and exact restoration.
`section_mixed` (4)	4 metadata faults + 2 Merkle-leaf faults + a damaged journal line; checks that everything is repaired anyway.

Block	What it does
`section_triggers` (5)	Damages a record + its leaf, then uploads again (before-write healing); then a `verified_get` on a damaged record (read healing).
`section_chunks` (6)	RS(4,2) store across 6 nodes; metadata faults + fragment faults + a node failure → `heal_all`; then 3 lost fragments → reported as unrecoverable.
`section_scale` (7)	Runs `benchmark/final_demo.py` once per system in separate processes (100k records by default) and prints the comparison.
`main`	Parses `--quick` / N, runs the sections in a temp folder, deletes it, prints the scorecard.

Core deduplication

`chunking/chunker.py`

Block	What it does
`DEFAULT_CHUNK_SIZE = 4096`	4 KB chunks by default.
`chunk_file(path, size)`	Reads the file in fixed-size pieces; each chunk gets `chunk_id = filename::chunkN::random8`, its index, bytes and size.

`hashing/hasher.py`

Block	What it does
`compute_hash(data)`	Returns the SHA-256 hex digest: the chunk's content address.

`hash_index/hashtable.py`

Block	What it does
`HashNode`	One linked-list node: key, value, next (`__slots__` saves memory).
`HashTable.__init__`	Bucket array of capacity 1024, size 0.
`_bucket_index`	First 8 hex characters of the key as an integer, mod capacity.
`insert`	Walks the chain; updates the value if the key exists, otherwise prepends a new node; resizes when load > 0.75.
`search` / `update` / `delete`	Walk the chain to find, overwrite or unlink the key.
`_resize`	Doubles the capacity and re-inserts every node.
`items`, `__len__`, `load_factor`	Iterate all records; count; current load.

`storage/cloud_storage.py`

Block	What it does
`CloudStorage`	Simulated cloud: one file per chunk named by its hash; `store_chunk` (skips the write if the file already exists), `read_chunk`, `exists`, `chunk_count_on_disk`.

`dedup_system.py`

Block	What it does
`ContentAddressableStorage.__init__`	Creates the hash table and the storage (plain or protected), the `file_manifest` (file → ordered list of chunk hashes), the `upload_log`, and an optional `index_listener` (the self-healing index).
`upload_file`	For each chunk: hash → `before_write` check → duplicate? increment `ref_count` and add to `referenced_by` : store the bytes and insert a new record → notify the listener (`on_upsert`). Saves the manifest and returns a report (chunks, unique, duplicates, bytes saved).
`reconstruct_file`	Reads the chunks in manifest order and writes the original file back.
`hash_table_dump`, `global_stats`	Debug dump; totals (logical bytes, bytes saved, dedup ratio).

Merkle / integrity layer

`merkle/record_digest.py`

Block	What it does
`record_digest(hash, entry)`	SHA-256 over `hash | chunk_id | storage_location | size | ref_count`. This is what the tree protects. `referenced_by` is excluded on purpose (it is unbounded bookkeeping).

`merkle/merkle_tree.py` (Phase 1, now the baseline)

Block	What it does
`_pair_hash`	Parent = SHA-256(left + right).
`MerkleTree.__init__`	Builds the levels bottom-up from the leaf list; an odd last node is paired with itself; stores the root.
`get_root`, `get_leaves`, `height`	Accessors.
`get_proof` / `verify_proof`	Inclusion proof: the sibling hashes from leaf to root, and the check that recomputes the root from them.

`merkle/integrity.py` (Phase 1, used by the conventional baseline)

Block	What it does
`_current_leaves`	Record digests of the whole index, sorted by hash.
`generate_root_hash`	Builds the tree and saves the baseline root and leaves.

Block	What it does
`verify_integrity`	Rebuilds the whole tree and compares the roots: detection in O(n).
`diagnose_changes`	Compares every leaf with the baseline: localization by full scan, O(n).

`merkle/bucket_merkle.py` (SH-CADI tree)

Block	What it does
Module docstring	Why we redesigned the tree, and the trust model (only `trusted_root` is trusted).
`EMPTY_BUCKET`, `CorruptionDetected`	Constant hash for an empty bucket; the exception raised when a write would build on a corrupt path.
`node_hash`	Internal node = SHA-256(`"node:"` + left + right) (domain separation).
`bucket_digest`	Leaf = SHA-256(`"bucket:"` + the bucket's record digests in sorted order).
`build_levels`	Builds every tree level from the leaves, O(B).
`depth_for(n)`	Smallest depth so buckets average ≤ 4 records.
`BucketMerkleTree.__init__`	Empty perfect tree with 2^depth leaves; `records` = bucket → {hash: digest}; sets `trusted_root`.
`bucket_of`	Top `depth` bits of the hash prefix = bucket number.
`root`, `total_records`	The stored (untrusted) root; record count.
`bulk_load`	Builds the whole tree from (hash, digest) pairs in O(n + B) and sets the trusted root.
`_verify_path(b)`	Recomputes leaf → root using the stored siblings and checks it equals `trusted_root`, O(log B).
`_refresh(b)`	Recomputes the leaf of b and its ancestors and moves `trusted_root` forward (legitimate updates only).
`upsert` / `remove`	Verify the path first (fail-stop), change one record, refresh one path.
`authenticate_bucket(b)`	Do the stored record digests match the leaf, and does the leaf chain to the trusted root?
`repair_leaves(buckets)`	Recomputes only the given leaves and their ancestors; does not touch `trusted_root` (the caller compares).
`derived_levels`	Recomputes every internal node from the stored leaves (used by detection).

`merkle/auditor.py` — Detect + Localize

Block	What it does
`attach`, `on_upsert`, `build_baseline`	Wire the auditor into uploads; build the initial tree from the index.
`_compute_live`	Hashes every live record into buckets and builds the "live" tree L.
`live_bucket`	Live digests of one bucket, saved from the last detect.
`detect`	Builds L (live) and D (from the stored leaves) and compares both roots with `trusted_root` → intact / data fault / tree fault, plus timing.

Block	What it does
`localize` → `descend`	Recursive top-down walk: equal nodes → prune the subtree; unequal at leaf level → flag the bucket. Records the trace for the demo.
`localize` → bucket loop	For each flagged bucket: leaf-only fault? ambiguous? otherwise diff live vs stored → changed / missing / extra. Returns the cost counters (nodes compared, records examined).
`audit`	Detect and, if damaged, localize.

Recovery layer

`recovery/journal.py`

Block	What it does
`entry_from_journal`	Turns a journal record back into a full index entry (`referenced_by` is restored as `[chunk_id]`).
`MetadataJournal.__init__`	Depth (same as the tree), optional directory (files) or in-memory dict, sequence counter.
`shard_of`	Same bucket function as the tree, so shard = bucket.
`_checksum`	First 16 hex characters of SHA-256 over the line body.
`_lines`, `_write_lines`, `_append`	Read, overwrite or append the lines of one shard (memory or file backend).
`_write` / `_parse`	Build a JSON line with sequence, hash, op, entry and checksum / parse it and reject it if the checksum fails.
`append_put`, `append_del`	Public API used on every index change.
`read_shard(b)`	Returns the newest value of every record in the shard, its history (newest first), and the line and invalid-line counts.
`shards`, `line_count`, `size_bytes`	Stats.
`tamper_last_record`, `wipe_shard`	Fault hooks: damage a line; delete a whole shard.

`recovery/healer.py` — the self-healing engine

Block	What it does
Module docstring	Recovery sources, and the rule: install only digest-matching candidates.
`UnrecoverableRecord`	Raised when a single-record heal finds no valid source.
`SelfHealingIndex.__init__`	Holds the hash table, tree, journal, auditor and optional `ChunkHealer`; registers read-triggered chunk repair.
`attach`	Becomes the CAS listener and gets access to the upload log.
`build_baseline`	Journals every existing record and builds the tree.
`before_write`	Calls `verified_get` before any upload changes a record.

Block	What it does
`on_upsert`	Appends the journal line, then upserts the tree.
`_refcounts`	Exact `ref_count` per hash = how many times it appears across all upload manifests.
`_resolve`	Tries journal history newest → oldest (and with the upload-log `ref_count`); returns the first candidate whose digest equals the expected one.
`_put`	Insert or update in the hash table.
`verified_get`	Authenticated read: checks the bucket, returns ok / removes spurious / heals just this record / falls back to a full heal.
`heal_all`	Metadata heal, then chunk heal; overall success flag.
`heal` → Detect, Localize	Calls the auditor; stops early if intact.
`heal` → RECOVER	Per flagged bucket: leaf-only → skip; ambiguous → rebuild the bucket from its shard; otherwise `_resolve` each changed/missing record and schedule extras for deletion. Nothing is modified yet.
`heal` → REPAIR	Applies the puts/deletes, installs the rebuilt bucket records, `repair_leaves(flagged)`.
`heal` → VERIFY	Per bucket: live digests == stored digests and the bucket authenticates; root == trusted root; optional full re-audit.
`heal` → return	A report with timings, records reconstructed/removed, sources, shards read, locality and success.

`recovery/baseline.py`

Block	What it does
`replay_journal`	Replays every shard into a brand-new hash table (what a conventional rebuild does).
`conventional_rebuild`	Replay + rebuild the whole tree; success only if the rebuilt root equals the anchor; locality = 1.0.

`recovery/erasure.py`

Block	What it does
`_init_tables`, `_EXP`, `_LOG`	GF(256) exp/log tables using the primitive polynomial 0x11D.
`gf_mul`, `gf_inv`	Multiplication and inverse in GF(256) via the tables.
`_scale`, `_xor`	Multiply a whole fragment by a constant (one `bytes.translate` call) and XOR two fragments (fast in pure Python).
`_invert`	Gauss-Jordan matrix inversion over GF(256).
`ReedSolomon.__init__`	k, m, n; builds the Cauchy parity matrix.
`encode`	Pads the chunk, splits it into k data fragments, computes m parity fragments.
`decode`	Any k fragments → invert the matching k×k sub-matrix (cached) → rebuild the data; the fast path when the k data fragments are all present.
`Replication`	r identical copies; tolerates r−1 losses (comparison scheme).

Block	What it does
`decode_verified`	Tries k-subsets until SHA-256(result) == the expected hash (catches silent corruption).

`storage/protected_store.py`

Block	What it does
`ChunkLost`, `ChunkCorrupt`	Fewer than k healthy fragments / no subset reproduces the hash.
`ProtectedChunkStore.__init__`	One folder per node (`node_0` … `node_{n-1}`); RS(4,2) by default.
`_pack` / `_unpack`	Fragment file format: 8-byte checksum, 4-byte chunk size, payload.
`write_fragment`, `read_fragment`	Write one fragment; read it and classify it as ok / missing / corrupt.
`healthy`, `chunk_health`	Collect the good fragments and their majority size; full health report including the content-hash check.
`store_chunk`	Encode and write fragment i to node i (same interface as `CloudStorage`).
`read_chunk`	Degraded read: decode_verified from the healthy fragments; triggers repair if any are missing.
`exists`, `chunk_count_on_disk`, `bytes_on_disk`	Stats.
`remove_fragment`, `corrupt_fragment(silent)`, `fail_node`, `remove_chunk`	Fault hooks.

`recovery/chunk_healer.py`

Block	What it does
`scrub`	Detect + localize for chunk data: checks every fragment of every chunk; lists damaged and lost chunks.
`heal` → recover	For each damaged chunk, decode_verified from the healthy fragments; no valid decode → unrecoverable.
`heal` → repair	Re-encodes and rewrites **only** fragments that are missing, corrupt or different.
`heal` → verify	Every fragment ok and the content hash matches; reports bytes read/written and locality.

Fault injection

`fault_injection/injector.py`

Block	What it does
`METADATA_KINDS`	The 5 metadata fault types.
`_log`, `_copy`	Ground-truth log of every fault; safe copy of a record before changing it.

Block	What it does
`modify_location`, `change_chunk_ref`, `modify_ref_count`, `delete_entry`, `insert_spurious`	Metadata faults.
`corrupt_merkle_leaf`	Overwrites a stored leaf (ancestors left stale, like bit rot).
`delete_chunk`, `delete_fragment`, `corrupt_fragment`, `fail_node`, `inject_chunk_faults`	Chunk-data faults.
`corrupt_journal_record`, `wipe_journal_shard`	Faults in the recovery information itself.
`inject_random(k)`	k faults on k distinct random records (+ optional leaf faults), seeded.
`affected_hashes`, `affected_leaf_buckets`, `affected_chunks`	Ground truth for checking localization.

Evaluation harness (`benchmark/`)

File / block	What it does
`systems.py` → `make_record`	Deterministic synthetic record i (SHA-256 key, 4 KB size).
`systems.py` → `ConventionalIndex`	Hash table + journal + Phase-1 tree; detect = rebuild the tree, localize = full diff, recover = replay the whole journal, verify = rebuild again.
`systems.py` → `TargetedConventionalIndex`	Same, but recovers only the records the full diff found (stronger baseline).
`systems.py` → `SHCADIIndex`	Hash table + journal + bucketed tree + `SelfHealingIndex`; `insert` updates the tree incrementally.
`systems.py` → `build`	Inserts n records, then seals (builds the tree / baseline root).
`normal_ops.py`	Upload throughput on 8,000 × 4 KB chunks (50 % duplicates) for three configs: plain hash table, + journal, SH-CADI.
`worker.py` → `trial_shcadi`, `trial_conv`	One fault trial: inject k faults, time each phase, check success and exact localization.
`worker.py` → `shard_loss`	10 faults + wipe one healthy journal shard; counts how many healthy records each system loses.
`worker.py` → `summarize`, `peak_rss_mb`	Median/mean/min/max of trials; peak memory (works on Windows and Linux).
`worker.py` → `main`	One (system, n) in a fresh process: build → lookups → clean verify → fault sweeps → rate sweeps → read-path → shard loss → inserts → JSON.
`run_all.py`	Runs every (system, size) worker as a subprocess and saves `results.json` after each one (supports `--resume`).
`report.py`	Turns `results.json` into `RESULTS.md` (all tables, ratios, limitations) and calls the plots.
`final_demo.py`	One system, N records, k faults, with step-by-step printout; used by demo section 7.

`visualization/plots.py`

Block	What it does
`_runs`, `_sizes`, `_med`	Helpers to pull medians out of `results.json`.
`recovery_vs_faults`	Recovery-phase time vs k, one panel per size.
`rate_sweep`	Locality and recovery time vs corruption rate (100k records).
`scalability`	Detection / localization / recovery vs index size.
`memory`, `normal_ops`, `localization`	Memory bars, upload throughput bars, localization work.
`make_all`	Draws all six plots into `benchmark_results/plots/`.

Tests (`tests/`, 66 in total)

File	What it covers
`test_phase1.py` (4)	Hash-table CRUD and resize; Merkle proofs; dedup + exact reconstruction; tamper detection.
`test_phase2a.py` (16)	Incremental tree = full rebuild; remove restores the old root; refused writes on a corrupt path; detection/localization for every fault type, multiple faults, leaf-only faults; tree stays in sync with uploads; chunk deletion is invisible to the metadata tree (by design).
`test_phase2b.py` (18)	Journal (memory and files); healing every fault type; ambiguous buckets; unrecoverable records reported; stale journal line rejected; localized vs full rebuild; no laundering of corruption on write; `verified_get`; system keeps working.
`test_phase3a.py` (20)	GF(256) arithmetic; RS decodes from every k-subset; replication; too few fragments; reads with up to m losses; checksums; overhead; byte-exact local healing; node failure; beyond tolerance; silent corruption; integrated metadata + chunk faults; read-triggered repair.
`test_phase3b.py` (8)	All three benchmark systems restore exactly and localize exactly; locality ordering; shard-loss scenario; harness helpers.